

Arquitetura da Camada 3 Motor QSV dos Folders

SOA/SOS · Sistema Operacional de Sustentabilidade · Soluções em Energia

A informação existe nos sistemas oficiais. O que falta é a engenharia
probatória que transforma dado público em tese defensável

julho de 2026

Hoje o sistema SOA tem **duas camadas**: **Camada 1** (dados e monitores — o banco energia_publica, ~143 mi linhas) e **Camada 2** (os 9 folders comerciais, apresentação). Falta a camada que liga uma à outra **com prova**. A **Camada 3 é o Motor QSV**: transforma cada **tese quantitativa** de folder em um **número com lastro auditável por terceiro**. É **um único motor** com **adaptadores por folder** — não um motor do zero para cada folder.

Princípio. Um folder comercial só é defensável se **cada número que ele cita puder ser reproduzido e verificado por um terceiro**. O QSV é o mecanismo que produz esse lastro: **score 0–3 por hora** por fonte independente e um **evidence pack imutável** (SQL reexecutável + hash + prova on-chain). **O QSV é o diferencial da SOA** — por isso toda a arquitetura o tem como base: **nenhum número chega ao folder sem passar pelo motor**.

O que este documento entrega ao Otávio (CTO). A arquitetura de referência do motor (pipeline, componentes, stack), o **QSV Manifest** que descreve o motor de cada folder, o **mapa QSV dos 9 folders**, o **roadmap de desenvolvimento** por fase, a mecânica de **auditoria por terceiro** e os **limites** — de forma eficiente, eficaz, segura e reexecutável.

Três invariantes de projeto:

- 1 · QSV-first — o motor é obrigatório, não opcional; a degradação é graciosa (flag qsvDegraded), nunca silenciosa.
- 2 · Reexecutável — todo número nasce de SQL versionado; o terceiro reproduz bit a bit.
- 3 · Imutável e ancorado — hash + Merkle + âncora on-chain tornam a prova resistente a adulteração.

A Camada 3 (Motor QSV) é a base de prova que sustenta todos os folders

Arquitetura em 3 camadas — o Motor QSV como base de prova de todos os folders

CAMADA 2

Apresentação

Baterias×Curtailment Contencioso Data Centers ESG Auditável Seguros Gato Solar IPTU Verde iCleanSolar Bitcoin Verde

9 folders comerciais vivos — cada um faz uma tese quantitativa (HTML lê feed + Fatos JSON)

↑ evidência auditável sobe (score + hash + link)

CAMADA 3 · MOTOR QSV (a terceira camada — o diferencial)

C1

Pre-commit
hash

C2

Timestamp
autoritativo

C3

Validação
orbital+twin

C4

Consenso
vizinhos H3

Σ

Score
0-3/hora

EP

Evidence
Pack

C5

Batch
on-chain

Off-chain first (v2.0): TimescaleDB · Merkle root diário · 5 TX/dia na Polygon · validação por hexágono H3 (custo marginal ~R\$0,001/gerador)

↑ dados oficiais sobem

CAMADA 1

Dados & Monitores

banco
energia_publica
~143 mi linhas

ONS
(geração,
constrained-off)

CCEE
(PLD)

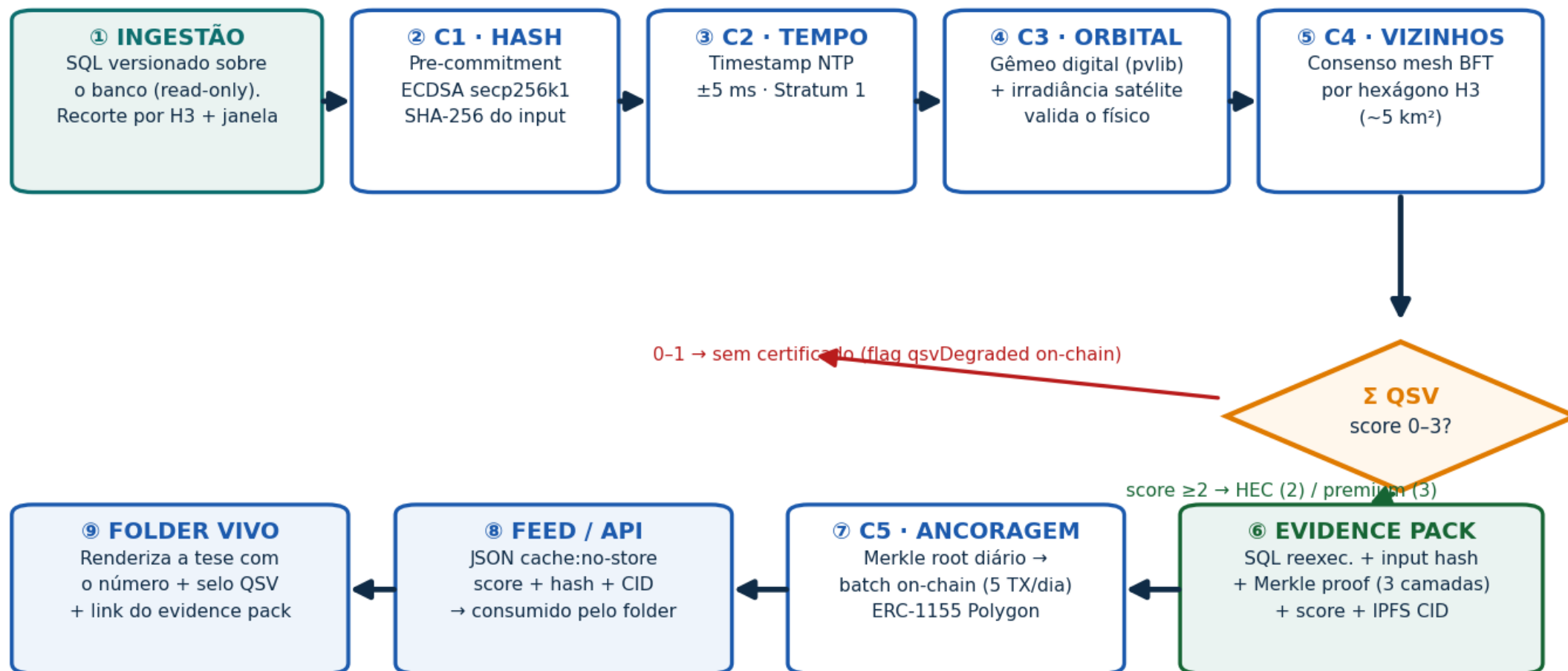
ANEEL
(SIGA/SIGET/
MMGD)

NASA POWER
(irradiância)

Monitores
Fatos_*.json
(regulatório)

Os dados oficiais (Camada 1) sobem para o Motor QSV (Camada 3), que os valida, pontua e empacota como prova: só então a evidência auditável (número + score + hash + link) sobe para os folders (Camada 2). O folder nunca cita um dado que não passou pelo motor.

Fluxograma do Motor QSV — do dado oficial à prova auditável por terceiro



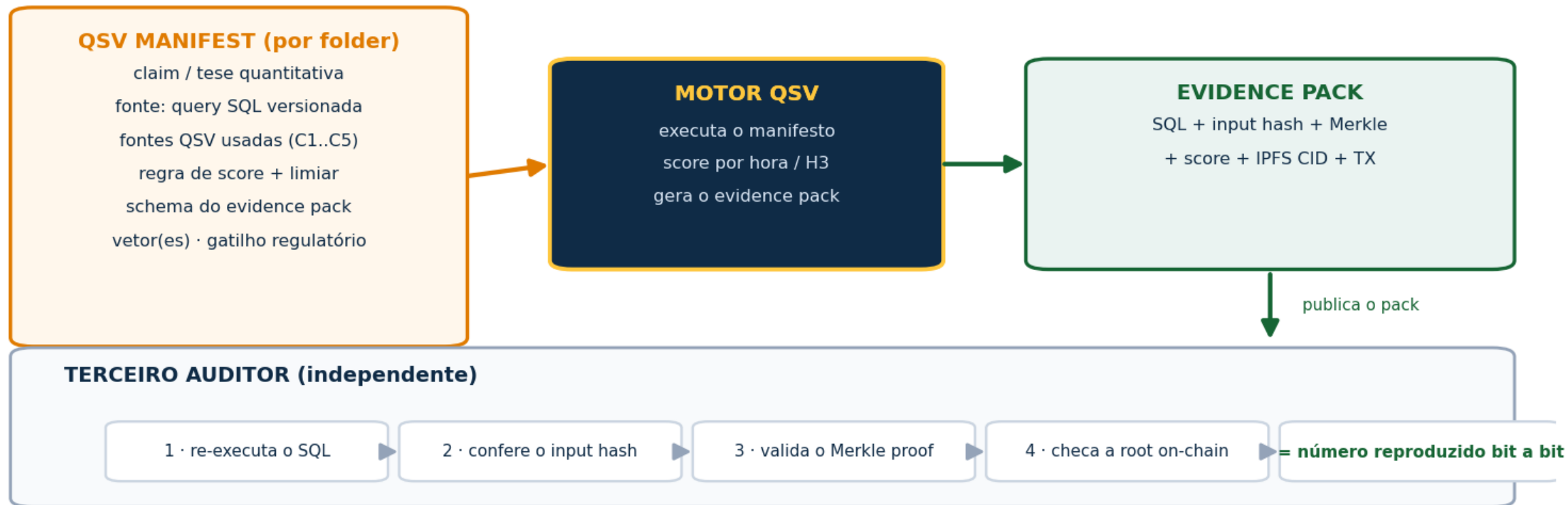
C1 congela o input (hash ECDSA secp256k1 + SHA-256) · **C2** carimba o tempo (NTP Stratum 1) · **C3** valida o físico (gêmeo digital pvlib + irradiância de satélite) · **C4** cruza com a vizinhança (mesh BFT por hexágono H3). O **score 0-3** decide: 0-1 sem certificado (qsvDegraded), 2 = HEC, 3 = premium. **C5** ancora o Merkle root diário on-chain.

Anatomia técnica e stack — o que o Otávio implementa

Componentes do motor, tecnologia sugerida e estado de maturidade

Componente	Função	Tecnologia sugerida	Estado
Ingestion workers	Extrai o recorte (H3 + janela) do banco via SQL versionado, read-only	Python + psycopg2 · SQL em git	Banco real
QSV Core	Motor de regras + scoring 0–3 por hora/hexágono; flag qsvDegraded	Python (rules engine)	A construir (TRL 7)
Gêmeo digital (C3)	Curva de geração esperada (teto físico) a partir de irradiância	pvlb · NASA POWER/Copernicus	A produzir
Camada cripto	Pre-commitment e integridade	ECDSA secp256k1 · SHA-256 · Merkle 3 camadas	Especificado
Timestamp (C2)	Tempo autoritativo da leitura/indício	NTP Stratum 1 (±5 ms)	Especificado
Âncora blockchain (C5)	Merkle root diário → batch on-chain (5 TX/dia)	Solidity/Foundry · HEC1155 ERC-1155 Polygon · web3	TRL 8 (18/18 testes)
Armazenamento	Séries temporais + pin do evidence pack	TimescaleDB · IPFS	Parcial
Evidence Pack gen.	Empacota SQL + input hash + Merkle proof + score + CID	Python	A construir
Feed / API	Serve score+hash+CID ao folder (JSON, cache:no-store)	FastAPI	A construir
Verifier público	Terceiro revalida o pack (re-run SQL + Merkle + root on-chain)	Python + web3 (endpoint/página)	A construir

Adaptador por folder (QSV Manifest) + verificação independente por terceiro



O Evidence Pack contém: (1) o **SQL versionado** que gera o número; (2) o **hash do input** (C1); (3) o **Merkle proof** de 3 camadas; (4) o **score QSV** por hora/H3; (5) o **CID IPFS** e o **hash da transação** de ancoragem (C5). **Segurança:** banco em usuário **read-only**; imutabilidade por hash+Merkle+âncora on-chain; **zero PII** e sem dado sensível em URL (LGPD). **Auditabilidade:** de posse do pack, o terceiro re-executa o SQL, confere o hash, valida o Merkle e checka a root on-chain — reproduzindo o número sem depender da SOA.

QSV Manifest — o adaptador de cada folder

Um arquivo declarativo por folder; o motor é genérico, o manifesto é específico

SOA/SOS · Camada 3 · Motor QSV

Cada folder comercial ganha um **QSV Manifest** versionado no repo (análogo aos `Fatos_*.json`, com CI gate). Ele **declara** o que o motor deve executar — sem código novo no motor. Plugar um folder novo = escrever um manifesto.

qsv-manifest.gato-solar-soa.json

```
{
  "folder": "gato-solar-soa", "vetores": ["V2","sHEC"],
  "claim": "geração observada acima do teto físico da potência declarada",
  "source_sql": "sql/gato_solar_reconciliacao.sql", // versionado em git
  "qsv_sources": ["C1","C2","C3"], // C3 = gêmeo digital pvlíb
  "score_rule": { "min_publish": 2, "premium": 3 },
  "h3_resolution": 7, "window": "hourly",
  "evidence_schema": ["sql_hash","input_hash","merkle_proof","score","ipfs_cid","anchor_tx"],
  "feed_out": "feeds/gato-solar.json", "gatilho_regulatorio": "CP 009/2026 · NT 107/2026"
}
```

Ciclo de vida: manifesto (semver) → CI gate valida schema/SQL → motor executa → evidence pack → feed → folder. Alterar a tese = novo manifesto v+1, com diff auditável. O **gate.yml** já existente no repo é o ponto natural para plugar a validação dos manifestos.

Mapa QSV dos 9 folders comerciais

Mesma máquina, nove teses — cada folder é um manifesto sobre o Motor QSV

Folder	Tese quantitativa	Dado (banco)	Validação QSV	Saída / Vetor
Baterias × Curtailment	BESS captura a renovável cortada	constrainedoff_eolica/fotovoltaica	C2+C3 por H3	curtailment horário · V3/V7
Contencioso Curtailment	Frustração renovável como lastro de tese	constrainedoff_detalhe_usina	C1+C2 por usina	sHEC (prova de frustração) · V4
Prova Horária Data Centers	Matching 24/7 renovável verificável	geração+carga+PLD horário	C1–C5 horário	HEC 24/7 · V2
Certificação ESG Auditável	Escopo 2 horário auditável	consumo × geração × fator emissão	C1–C5 → relatório	HEC/sHEC + relatório · V6
Seguros de Renováveis	Risco de curtailment precificável	constrained-off histórico + geração	C3 + estatístico	data pack de risco · V8
Gato Solar	Injeção acima do teto físico declarado	mmgd_* + nasa_power_*	C3 (gêmeo digital)	reconciliação (RPD-GFP) · V2
IPTU Verde Auditável	Telhado/GD tornado visível e auditável	mmgd_* + cadastro municipal	C3 gêmeo digital	prova de telhado · sHEC
iCleanSolar	Queda de geração = sujeira/falha detectável	geração/telemetria + vizinhança	C4 vizinhos + anomalia	evidência de performance · B2C
Bitcoin Verde	Mineração lastreada em energia limpa horária	geração + consumo carimbados	C1–C5 horário	prova verde · creators

Linha destacada = Gato Solar (arquitetura detalhada em documento próprio). Todas as saídas nascem de SQL versionado e evidence pack.

Roadmap de desenvolvimento — para o CTO

Sequência de construção, dependências e estado de maturidade (TRL)

Fase	Entregável	Depende de	Estado
0 · Base (feito)	Banco real (energia_publica); contrato HEC1155 (18/18 Foundry AMOY); QSV conceitual	—	Concluído · TRL 8 contrato / TRL 7 QSV
1 · Core off-chain	QSV Core (scoring 0–3) + Evidence Pack generator + schema do Manifest; 3 folders-âncora (Gato Solar, Baterias, Data Centers)	Fase 0	A iniciar
2 · Físico (C3/C4)	Serviço de gêmeo digital (pvlib) + consenso de vizinhos H3	Fase 1	A iniciar
3 · Ancoragem (C5)	Merkle diário → batch on-chain produzizado + pin IPFS	Fase 1 · contrato pronto	Parcial (contrato TRL 8)
4 · Adaptadores + feed	Manifestos dos 9 folders + Feed API + Verifier público	Fases 1–3	A iniciar
5 · Operação	Observabilidade (qsvDegraded, cobertura, freshness) + CI gate dos manifestos	Fase 4	A iniciar

Ordem recomendada: Fase 1 sobre os **3 folders-âncora** (que já têm tese madura e dado no banco) prova o motor end-to-end antes de escalar para os 9. O contrato on-chain (Fase 3) já está em TRL 8 — o caminho crítico é o **QSV Core + Evidence Pack** (Fase 1).

Complementos recomendados

O que eu acrescentaria para deixar a arquitetura mais robusta e auditável

- **QSV Manifest versionado + CI gate**

Um manifesto por folder no repo (semver), validado pelo gate.yml já existente — mesma disciplina dos Fatos_*.json. Mudança de tese vira diff auditável.

- **Verifier público**

Uma página/endpoint onde o terceiro cola o evidence pack e revalida sozinho (re-run SQL + Merkle + root on-chain). Materializa a promessa de 'auditável por terceiro'.

- **Observabilidade do motor**

Painel com cobertura (%), freshness por fonte e taxa de qsvDegraded por folder — exposto como a 3ª camada do Catálogo (status QSV de cada folder).

- **Degradação graciosa obrigatória**

Score 0–1 nunca vira número publicado; grava qsvDegraded on-chain. O folder mostra 'sem lastro' em vez de um número frágil.

- **Governança de versão + assinatura**

Cada evidence pack e manifesto assinado (chave do CTO). Semver e changelog por folder; a Tabela de Precedência QSV governa exceções regulatórias.

Estado de maturidade

Banco energia_publica é real (~143 mi linhas). Contrato HEC1155 em TRL 8 (18/18 testes Foundry, fork Polygon AMOY). O QSV Core, o gêmeo digital produtizado, o Evidence Pack, o Feed e o Verifier ainda serão construídos (Fases 1–5). Plataforma HEC pré-operacional (schema vazio, 0 tx em mainnet).

Dependência de dados observados

Fontes que exigem medição por UC (gato solar, iCleanSolar) dependem de convênio com a distribuidora; sem isso, o motor opera em modo triagem sobre o cadastro.

Incerteza do gêmeo digital

Quando faltam orientação/inclinação, assume-se geometria típica → margem de erro declarada. O QSV pontua a confiança; não substitui vistoria onde a regra a exige.

Segurança e LGPD

Banco read-only; imutabilidade por hash+Merkle+âncora; zero PII e sem dado sensível em URL. Papel de prova, nunca de acusação — quem autua é o regulador/distribuidora.

Propriedade intelectual

Três pedidos de patente depositados no INPI — comunicar como 'tecnologia com patente depositada' (pedidos em análise); evitar afirmar concessão.



SOA

SOLUÇÕES EM ENERGIA

Obrigado.

Luiz Henrique Limonta
CEO
limonta@solaroneaccount.com
+55 (44) 99972-0477

Otávio Augusto Ferrari Pedro
CTO
pedro@solaroneaccount.com
+55 (44) 99764-3333

SOA/SOS · Sistema Operacional de Sustentabilidade · Soluções em Energia

Solar One Account Ltda · CNPJ 65.312.574/0001-40 · solaroneaccount.com

Confidencial